

A primer on logical causality

Brian McPeak

October 2, 2026

1 Intro

Can mathematical facts ever be said to “cause” each other? If A causes B , what does this mean, and how is it different from implication?

I don’t know the answer, but this document will introduce some basic concepts that might be useful for this question. We will start with a review of causal graphs and how to infer causality from joint probability distributions. Then we will get into a few different ways of thinking about uncertainty over mathematical or logical facts. We’ll end with some examples that a good account of logical causality might need to explain. Here is a longer list upfront – some of these are discussed at more length below.

- When do $2|p$ and $3|p$ both cause “ p is prime”?
- Should a statement A cause $A \wedge B$? What about $A \vee B$? Claiming both of these always hold will get you into some trouble.
- When do special cases cause general cases, e.g. “Fermat’s last theorem holds for $n = 10$ ” causes “Fermat’s last theorem”
- Suppose A causes B and $B \iff C$. Is there necessarily any relationship between A and C ?
- Different algorithms with the same outputs, e.g. merge sort, bubble sort, and “the sorted list is...”
- product of three numbers abc – what is its causal relationship with ab and bc ?
- The 100th and 101st digits of π are independent. But are they common causes of anything?
- The Fermat primality test gives “good evidence” that a number p is prime. What is its causal relationship with the statement p is prime.
- Does the value of a hash H at x cause (average value of H).
- Suppose P is a slow random permutation. Knowing $P(x_1)$ tells you something about $P(x_2)$ (they must differ). Are different values of $P(x)$ causally related?
- Represent an algorithm with a computational graph. When is it a causal graph? What causal relationship do numbers showing up in intermediate stages of a calculation have

to each other?

- Does the state of a chaotic system at t_1 cause the state at $t_2 > t_1$? Why not the reverse, if the dynamics is reversible in principle? This points to something about computational complexity?

2 Causal graphs

A *causal graph* is a directed acyclic graph (DAG) plus a probability distribution that satisfies Markov factorization. Markov factorization is the following property of the probability distribution:

$$P(v_1, \dots, v_n) = \prod_i P(v_i | \text{pa}_i) \quad (1)$$

This means that the distribution of the values at each node, v_i , depend only on the parents of that node, pa_i .

Any probability distribution can be written in a factorized form using the chain rule, *e.g.* $P(a, b, c) = P(a)P(b|a)P(c|a, b)$ When written in this way, the nodes are ordered and each node depends on every node before it. A complete DAG, which has an edge between every pair of nodes, can fit any distribution. Thus the information in a causal graph is really about which edges are not present. It says: each node *only* depends on its parents.

A given distribution will not generally have a unique graph. For instance, when we have two variables A and B , the graphs $A \rightarrow B$ and $B \rightarrow A$ are both consistent with any distribution. And we can always add edges to any causal graph, provided the result remains acyclic. Thus we will sometimes be interested in the minimal causal graph.

Conditioning on various nodes can make previously independent nodes dependent, and vice-versa. Roughly, conditioning on a collider (like B in the subgraph $A \rightarrow B \leftarrow C$) makes the parents (here, A and C) correlated. The opposite happens for chains ($A \rightarrow B \rightarrow C$) and forks ($A \leftarrow B \rightarrow C$) – in those cases, A and C are dependent before conditioning on B but independent after¹

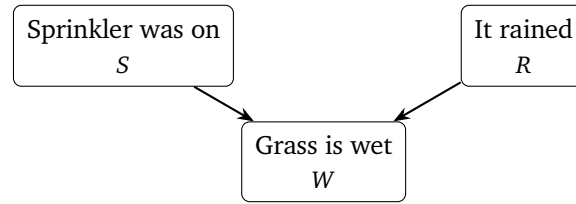
Causal graphs are also often said to be endowed with a “do-operator.” This operator takes a graph and returns a new graph where you have:

- set a particular node to a particular value
- assume that value in children nodes
- cut all arrows coming into that node

¹The actual rule to tell if A and C are independent after conditioning on some subset Z is called d-separation. You consider all paths between the nodes, ignoring arrows. Chains and forks are open by default but blocked if the middle node $M \in Z$. Colliders are blocked by default but open if M or any of its descendants are in Z . A path is open if all of its intermediate nodes are open. Two nodes are independent given Z if all paths between them are blocked.

The idea is that you enforce some sort of intervention, and then the new graph encodes the consequences of that. So you assume the value you chose in later nodes. But you cut the incoming arrows because you don't want to use the fact of your enforced value to infer anything upstream. Importantly, this is different from conditioning on the value you choose. Conditioning might introduce correlations between the nodes upstream. *do* cannot.

example. Let's illustrate these ideas with a simple example. Suppose two causes of the grass being wet this morning are (1) the sprinkler turned on last night, and (2) it rained last night. This could be encoded in the graph



Now let's suppose that we have some set of probabilities for all of these:

$P(S=1)$	$P(R=1)$			
0.5	0.2	S	R	$P(W=1 \mid S, R)$
		0	0	0.05
		1	0	0.80
		0	1	0.90
		1	1	0.99

We can see that S and R are *a priori* independent. However, if we condition on the grass being wet, then they both become more likely, and fail to remain independent. This is clear from the conditional probabilities:

Quantity	Value
$P(R=1)$	0.20
$P(R=1 \mid S=1)$	0.20
$P(R=1 \mid W=1)$	0.36
$P(R=1 \mid W=1, S=1)$	0.24

However, the *do*-operator is different from conditioning. It's more like I got out the hose and watered the grass right now. I cut the incoming arrows because after $do(W = 1)$, I know that the grass is wet because I hosed it down, so I shouldn't use the fact that it's wet to infer anything about last night. If W had any downstream nodes, then I could simplify all of their distributions by assuming that $W = 1$.

3 Logical uncertainty

In the example above, causality is a notion that arises from the probability distributions describing our beliefs or predictions about various facts. The point of this particular document is to understand how causality works for mathematical or logical statements. An important question is: what does it mean to be uncertain about such a statement?

Of course, I can easily say “I think there’s a 90% chance that $P = NP$ ”. But how do we formalize that? I think the answer at this point is that “we don’t have a perfect answer.” This would be a good thing to try to clarify. I’ll review a few ideas people have had in the past below. They might serve to illustrate the *sorts* of thing one might look for.

3.1 Logical induction

One idea is what’s called logical induction, where traders use polynomial time algorithms to trade on whether different mathematical statements are true or false. The credences are given by the market prices of those statements. As a prerequisite we should start with a somewhat simpler form of induction:

Solomonoff induction. Consider the following question: I give you 10 bits, 1010101010. What is the next bit?

It’s appealing to say that 1 is more likely because I gave you a simple repeating pattern. However there are an infinite number of sequences compatible with the next bit being a zero or a one. So *why* should 1 be more likely? And how much more likely, exactly?

Solomonoff induction basically answers this question by saying that 1 is more likely because there is a “simpler” reason for it to be 1, by which we mean a simpler computer program where 1 is the next bit of output. All sequences of numbers are not equivalent – instead, we sum over all Turing machines that print out that particular sequence, weighted by $2^{-|p|}$, where $|p|$ is their length. Formally we define $M(x)$, the prior that describes how likely a sequence of bits x is, by

$$M(x) = \sum_{p: U(p) \text{ starts with } x} 2^{-|p|} \quad (2)$$

Here U is a Universal Turing Machine, which takes a Turing machine p as input and prints its output. So this formula says that the likelihood of x is higher when there are very simple computer programs that print x . The fact that this number is always finite follows from Kraft’s inequality (to ensure this is true, we impose the additional requirement on U that it is prefix free, meaning that no valid program is a prefix of another).

Okay, so how do we use this to determine the probability that the next digit is a 1? The answer is Bayes’ theorem! The probability that the next digit is 1, given that the first digits are the same as x , is the probability that we draw x and then one on the next digit, over the

probability that we draw x . In equations,

$$P(1|x) = \frac{M(x1)}{M(x0) + M(x1)} . \quad (3)$$

Logical induction. Solomonoff induction allows us to talk about uncertainty in, for instance, the next bit of a sequence. But now suppose we are interested in the question “what is the $10^{100\text{th}}$ digit of π ?” How could we come up with a probability distribution over the 10 possible answers?

One tool for doing this is called *logical induction*. The setup is the following:

- a Deducer systematically proves statements. On day n the set of all statements proven to date is D_n
- a Market Maker sets prices of all statements, proven and unproven
- Traders buy and sell shares in statements, using strategies that take a polynomial amount of time in the number of days

At any day n , there are many different worlds that are consistent with all known facts D_n . In a particular world W , one can compute the value of a particular trader’s holdings by the formula

$$V_n(W) = c_n + \sum_S s_n(S)W(S) \quad (4)$$

where c_n is the cash the trader has from buying / selling positions. The sum is over all statements S . $s_n(S)$ is the number of shares in S the trader holds on day n , and $W(S)$ is 1 if S is true in world W , and 0 if it’s false.

The market maker is called a *logical inductor* if it can’t be exploited by any polynomial time trader. A trader exploits the market if the set of all its possible values is unbounded above but bounded below. That is, it exploits the market if there is any possible world where its value goes to ∞ but no world where its value goes to $-\infty$.

3.2 Multicalibration

Another fun idea for defining logical uncertainty is *multicalibration*. The idea is that, given a big class of yes / no mathematical questions, there shouldn’t be any simple subset that you are systematically biased on.

Let’s be more precise. First we consider some reference class C of mathematical yes or no questions – for instance, “is the n^{th} digit of π 7”, where n is drawn uniformly between 10^6 and 10^7 . Each question has a single answer that might be hard to compute. We’ll call the questions x and the answer $y(x)$. So we know that $y(x) \in \{0, 1\}$.

Now we want to consider some credences, $p(x)$. Recall that a distribution is *calibrated* if

things it predicts with probability p happen a fraction of time p . That is,

$$| \mathbb{E} [(y(x) - \nu) | p(x) = \nu] | < \alpha \quad (5)$$

Here α is the tolerance, which should be small. So the definition says that the expectation values of facts with credence r should be about r .

Now we take our set of mathematical facts and choose a set S of efficiently computable subsets. Then a set of credences $p(x)$ for facts in C is multicalibrated with respect to S if it is calibrated for all $s \in S$.

The definition we are going to use is the following:

Definition. Let $\mathcal{D}$ be a distribution over a class C of yes/no questions, with true answers $y : C \rightarrow \{0, 1\}$. A predictor $p : C \rightarrow [0, 1]$ taking finitely many values is (S, α) -multicalibrated if for every test set $s \in S$ and every value ν in the range of p ,

$$\left| \mathbb{E}_{x \sim \mathcal{D}} [\mathbb{1}_s(x) (y(x) - \nu) | p(x) = \nu] \right| \leq \alpha.$$

This is the usual definition plus an indicator function $\mathbb{1}_s(x)$ to ensure that x is in the set s . Note that this is not the same as

$$\left| \mathbb{E}_{x \sim \mathcal{D}} [(y(x) - \nu) | p(x) = \nu, x \in s] \right| \leq \alpha.$$

as the two definitions differ by a factor of $\mathbb{E}[\mathbb{1}_s(x) | (p(x) = \nu)]$ —effectively the size of the group s . We will see why it is useful to take this definition later. Both definitions, and more have been considered in the literature; the paper that introduced multicalibration, 1711.08513, had a slightly more complicated definition that was more like the latter.

We require that $p(x)$ takes finitely many values so that there is a finite number of values where $p(x) = \nu$ —if we want to define $p(x)$ to take a continuum of values, we can use bins and require that this holds for $p(x) \in [\nu - \delta, \nu + \delta]$.

Different definitions of “efficiently computable” might be sensible. For us, we will take it to mean that there is a Boolean circuit with at most T gates that outputs 1 exactly on the questions in c .

example 1: primality. Our set C consists of questions “is q prime,” for q every number between 10^6 and 2×10^6 . The rate of primes in this range is 7.04%. So the credence $p(q) = .0704$ is calibrated over the entire set.

But consider the fact that there is a simple 1-gate test for evenness (which is simply NOT of the last bit). If we define $s_2 \in S$ to be even numbers, then $p(x)$ is *not* calibrated on s_2 . A better predictor is zero on s_2 and .1408 on its complement.

This second predictor is calibrated on s_2 and s_2^C but not s_3 which are the numbers divisible by 3. So we should define a 3rd predictor that is 0 when a number is in s_2 or s_3 and .211 otherwise. And so on.

An important fact is that it takes more gates to check divisibility by higher primes (is this actually always true?). So once I fix a budget T , there is some maximum prime I need to worry about, p_n , and I will only be multicalibrated with respect to $S = \{s_2, s_3, s_5, \dots, s_{p_n}\}$

example 2: digits of π . Now consider the question: “is digit n of π a 7?” where $n \in [10^6, 10^7]$. Claude estimates that it takes $10^9 - 10^{10}$ gates to compute digits of π in the $10^6 - 10^7$ range. It doesn’t matter what the actual answer is – let’s just call it T^* .

For $T \ll T^*$, .1, the natural multicalibrated predictor is .1 for every n . Or at least, this holds empirically but isn’t proven.

Once T gets closer to T^* then you can start computing the answer, so the proper distribution becomes exact for digits that can be computed, and .1 otherwise.

One concern is that your algorithm could include lookup tables. In other words, I could consider a circuit that just encodes some 100 or a 1000 or $\sim 10^6$ values of n where the n^{th} digit is 7.

One option is to just accept this. After all, the indicator function in the definition that we said we’d come back to says that for a small set, we don’t need to be as calibrated. If we just have a lookup table for the 100 values of n but then we effectively divide α by the probability that x is in s , then we get a huge tolerance. And if we have a large number of values then we need a large number of gates to encode all of them. It might be somewhat lower than T^* , but that’s fine – we just redefine T^* to the lookup table circuit.

Another option is to, instead of circuits, consider programs with fixed input size. Then the number of gates becomes analogous to the runtime. Here a lookup table would correspond to a program that is very long (in bits, not runtime). So we could rule out lookup tables by putting a cap on both runtime and description length.

I don’t know which of these will be the most fruitful.

Finally, just using huge class sizes might work for some problems. However for digits of π , both the lookup table size and the actual algorithm scale linearly with n .

example: square numbers. Now consider: “is n a perfect square for $n \in [10^6, 2 \times 10^6]$ ”. There are 415 squares in this range, so the naive estimate is .000415. Now note that $p(x) = 0$ is multicalibrated for any $\alpha > .000415$. This is something to watch out for – multicalibration is not a good way to get sensible credences for events that are rarer than α .

example: twin primes. Question: are n and $n + 2$ prime, for $n \in [10^6, 2 \times 10^6]$?

Now suppose that we assume independence. Then each number has a $1/\ln n$ chance to be prime, and we’d estimate that there are

$$\sum_n \frac{1}{\ln n^2} \approx 4961 \tag{6}$$

So our naive estimator puts have a .005 chance for any n . The number of twin primes in that range (per Claude) is 6702. So even on the full set, we aren't calibrated.

Recall that for the single primes, we had a sequence of distributions, let's call them $p_1, p_2, \dots$ that were calibrated when we look at the whole set only, the whole set plus s_2 , the set of even numbers, and so on. The calculation above comes from taking p_1 for each prime, plus the assumption that each is independent. However, if we use p_2 , which is about 14% on each odd number and zero on the evens, and assume independence, then we estimate 9,922 twin primes. This is because if n is odd then $n + 2$ is odd, so assuming that one is odd multiplies the chance that the other is prime by a factor of two. Then subsequent refinements lower the value, basically because if n is not equal to $0 \bmod 3$, then $n + 2$ is slightly more likely to be $0 \bmod 3$. And so on. The result of doing this for increasingly large subsets, is encoded in the following table:

Estimator	Conditions on residues mod	Predicted twins	Predicted / actual
p_1	— (PNT only)	4961	0.74
p_2	2	9922	1.48
p_3	2, 3	7441	1.11
p_4	2, 3, 5	6976	1.04
p_5	2, 3, 5, 7	6783	1.01
p_6	2, ..., 11	6715	1.00
p_7	2, ..., 13	6668	0.99
p_9	2, ..., 19	6622	0.99
p_∞ (Hardy–Littlewood)	all p	6551	0.98
Actual		6702	1

This procedure also hints at how we'd find the joint distribution for n is prime and $n + 2$ is prime. This is important for figuring out the causal graphs, where we require Markov factorization of the joint distribution. So if A and B are nodes, then we need to consider $P(A \wedge B)$, $P(A \wedge \neg B)$, $P(\neg A \wedge B)$, and $P(\neg A \wedge \neg B)$.

Our estimators give a non-zero correlation R , where

$$R = \frac{P(A \wedge B)}{P(A)P(B)},$$

If $P(A)$ is the chance that n is prime, and $P(B)$ is the chance that $n + 2$ is prime, both drawn randomly, then p_i has correlation 1, 2, 1.5, 1.41, ... $\rightarrow$ 1.32

This means that unconditionally, A and B are positively correlated. Thus they cannot be the roots of our causal diagram. However I'm having trouble determining what the exact causal diagram should be, since statements like $q|n$ and $q|(n + 2)$ are not independent. So I'm going to stop for now, but figuring this out would be a good direction later.

issues and further improvements. An obvious issue is that multicalibration only applies to aggregates. If, for instance, in the first example with prime numbers I have a credence of 80% that 1,000,002 is prime, multicalibrating can't really tell me that is bad because this won't affect any test on a big set much.

One way to fix this might be to use α as a measure of how good a distribution is – priors that are multicalibrated to within a smaller α might be better. The problem with this is that α is a max error – we require that no subset and $p(x) = \nu$ has error above α . Maybe a better version would be a measure that averages α over subsets and values ν . This can get us into proper scoring functions and loss minimization; see e.g. [arxiv:2304.09424](https://arxiv.org/abs/2304.09424).

4 Examples

Now let's put aside, for the moment, where our uncertainties about mathematical facts come from, and consider some examples which a good theory of logical causality should be able to handle.

4.1 primality tests

Let's suppose that I draw an integer p from uniform random distribution between 10^6 and 2×10^6 . Consider the three facts:

- (A): 2 divides p
- (B): 3 divides p
- (C): p is prime

Given this particular statement of the problem, it is clear that (A) and (B) are approximately independent. However both are correlated with (C). The simple structure compatible with this is $A \rightarrow C \leftarrow B$.

Now, this example is underspecified in various ways. In particular, I didn't tell you what my credences were for (A), (B), (C), or (C) given (A) and / or (B). Since p is drawn randomly, before I do any computation, the natural credence for (A) is 1/2 and (B) is 1/3. What should I take as my base rate for (C)? If I didn't know anything about prime numbers, I might just assume that it's 1/2, since it's just a random proposition (or, I could adjust this with some prior based on how often a random proposition presented in a problem could be true). But then I have no reason to expect that the answer is correlated with (A) and (B).

If I knew the prime number theorem, I could estimate that there are 70,427 primes between 10^6 and 2×10^6 (which is very close to the real number, 70,435). This means roughly that a number in that interval has a 7% chance of being prime.

Regardless of what estimate we use for the number of primes, it's true that none of them are even or divisible by 3. So if I learn $\neg(A)$, for instance, then the probability that it is prime should double. If I learn $\neg(B)$ it goes up by a factor of 3/2. And if I learn both, it goes up

by a factor of 3. So any estimate of the number of primes is enough to figure out a sensible distribution with the causal properties we suggested.

Another point: it was important that we choose the prime from a uniformly distributed distribution. Suppose I choose randomly from a set of numbers that all equal either 0 or 1 mod 6. Then clearly (A) and (B) are no longer independent, and the causality diagram cannot be the (C) collider that we had above. So *at a minimum*, we need to know how the inputs to the calculation are going to be drawn.

Other questions:

- does $6|p$ cause p is prime?
- what about $q|p$ when $q > \sqrt{p}$?

4.2 Special cases

A big class of examples concerns the relation between a fact and its special cases. For instance, suppose $f(x)$ takes integers and returns true or false. The general case G is something like $f(x) = \text{true}$ for all x . A special case S is something like $f(3) = \text{true}$.

I think it's natural to suggest that special cases cause general facts because a basic test of causality is: if there is any A that is correlated with G but not S then G can't be upstream of S , and vice versa. Now suppose I find a counterargument to G . This implies that G is not true but it is (or maybe could be, depending on the type of counterargument) independent from S . So S should be upstream. For example:

- Fermat's last theorem when n has an even number of digits $\rightarrow$ Fermat's last theorem

I might find a counterexample to FLT but this doesn't tell me if it's true for a totally different instance. (Or there may be some cases where it does)

Heuristics. When we have a heuristic, we can think of it as a cause of the special case and the general case. This is because there can be various facts that weigh on whether a theorem is true without affecting whether the heuristic is true.

- Heuristic argument for Goldbach's conjecture $\rightarrow$ Goldbach's conjecture holds for 452938452 $\rightarrow$ Goldbach's conjecture

This is because I could disprove the special case – maybe Goldbach actually doesn't hold for some particular number but this does not invalidate the heuristic. Again, this reasoning seems to depend on exactly what kind of heuristic we have in mind, and it could be fruitful to try to enumerate different kinds of heuristics to see if the logic holds.

4.3 Logical consequences

Consider disjunction, *i.e.* logical *or*. When does a true statement cause a disjunction?

- $A \rightarrow A \vee B$

This seems like a good case for causality. In particular I could get some information about B , which could make me more likely to believe the latter but has no bearing on the former.

Next consider logical *and*:

- Conjunction, $A \rightarrow A \wedge B$.

This seems similar. After all, I can imagine that learning that B is true at least makes me somewhat more likely to believe A and B , but not only A .

However this leads to a problem. Consider that $A \rightarrow (A \vee B) \rightarrow (A \vee B) \wedge (A \vee \neg B) = A$.

This seems like a big problem. One could treat A and $(A \vee B) \wedge (A \vee \neg B)$ as truly distinct nodes on the causal graph, but this seems pretty messy and unappealing. I think it's better to take away from this that you can't assume that statements always cause both their conjunctions and disjunctions. Maybe it's true for "suitably simple" statements or maybe using a particular model of uncertainty will clarify when it might be the case.

4.4 $\text{Tr}(ABC)$

Suppose we want to compute $\text{Tr}(ABC)$. There are different ways of doing this, including

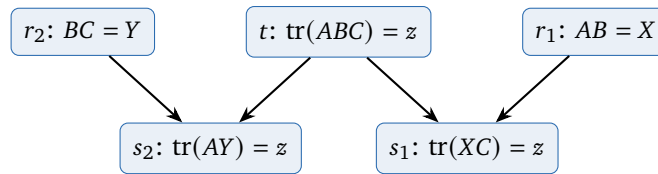
- compute $X = AB$, and then compute $z = \text{tr}(XC)$
- compute $Y = BC$, and then compute $z = \text{tr}(AY)$

The question is: what is a sensible causal graph for these various quantities?

I thought a lot about this and it's a sticky question. The thoughts below are probably totally on the wrong track. First, assume that we have 5 statements:

1. $r_1: X = AB$
2. $r_2: Y = BC$
3. $s_1: z = \text{tr}(XC)$
4. $s_2: z = \text{tr}(YA)$
5. $t: z = \text{tr}(ABC)$

Then I just came up with some probability distributions for these Booleans. For instance, there are clearly 32 possible assignments of true / false among them. However only 18 are compatible with logical facts like $r_1 \wedge s_1 \implies t$. One could make each of these assignments equally likely, i.e. give a 1/18 chance to each one. Then you can look at which causal graphs are consistent with that distribution. The answer turns out to be that the minimal graph that is symmetric under $(r_1, s_1) \longleftrightarrow (r_2, s_2)$ is

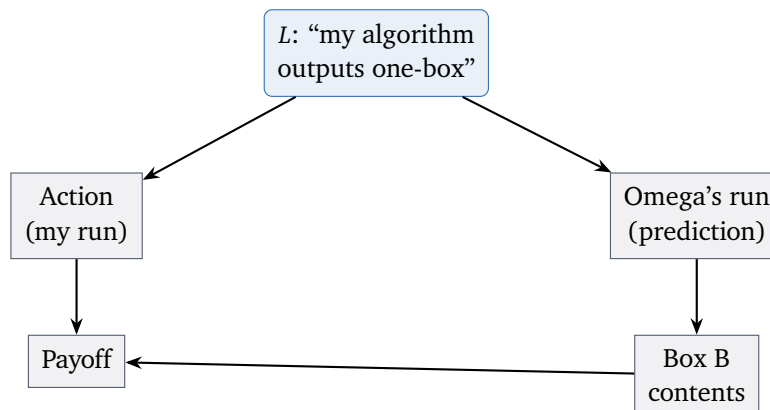


This is well and good but I'm not sure what it teaches us – it's a totally arbitrary distribution. I think that a more fruitful direction is to try to understand more about the sources of uncertainty.

A Decision theory examples

Some previous thinking about logical causality, such as [this post](#), has focused on the question: how do logical facts interact with events in the world? I started by considering these but now I think it's beside the main point, so I'll stick these examples in an appendix.

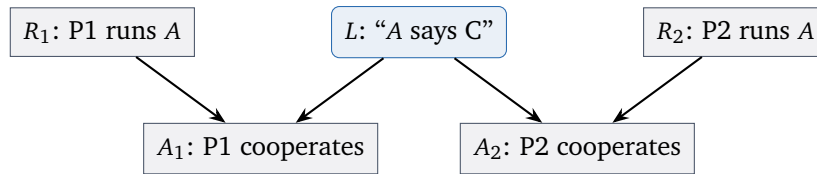
Consider first Newcomb's paradox. Omega predicts by running an algorithm *exactly equivalent* to yours (same input–output behavior on this problem). Box A holds \$1,000; box B holds \$1M iff Omega's run outputs "one-box".



- Because the algorithms are exactly equivalent, L is the output of a shared computation: this is a pretty clear case.
- an interesting change you can make is to suppose that Omega first proves that algorithm $\tilde{A}$ has the exact same output as algorithm A . Then it runs $\tilde{A}$ to determine whether to fill the box. Then we probably should have A outputs 1 and $\tilde{A}$ outputs 1 as separate causes.
- we can go further and suppose that Omega runs an algorithm which he tests and he believes it is the same as mine in 99% of cases. Then what does the causal diagram look like?

A.1 Prisoner's dilemmas

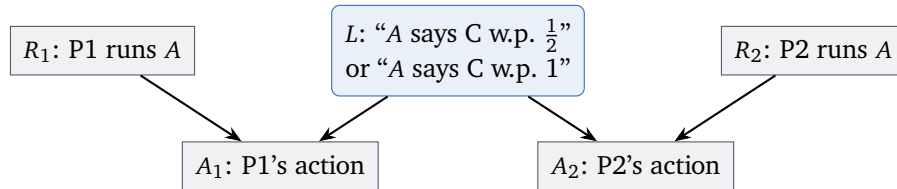
Twin prisoner's dilemmas are another interesting case. First suppose that P1 and P2 each run algorithm A , and A says to cooperate. Then we have a diagram like this:



Next we could add various sources of uncertainty: the algorithm could be random, or there could be uncertainty about what the algorithm is. These differ because randomness might lead the two runs to differ between the twins, but uncertainty won't. Let's suppose that P1 and P2 each run algorithm A. There are two things that A might be, with 50% odds each:

- cooperate with probability $\frac{1}{2}$;
- cooperate with probability 1.

Each player samples with their own independent randomness.



If $L = \frac{1}{2}$, the actions are two independent fair coins; if $L = 1$, both are C. Either way $A_1 \perp\!\!\!\perp A_2 \mid L$. Here we've combined two sources of randomness – A itself is uncertain (randomness of L) and A is (possibly) a stochastic algorithm. We could make more examples by separating these.